GRASP: Claim-Level Hallucination Detection from Attention

Project report — BCSE306L Artificial Intelligence, Topic 16: Uncertainty-Aware Abstention in LLM Applications

> **Method:** GRASP (**GR**ounding, **A**ttention **S**ink, dis**P**ersion), version 2.
> **Benchmark:** MUCH (arXiv 2511.17081): 4,873 generations, 20,751 claims, 4 languages, 4 open LLMs.
> **Main result:** On MUCH's human-annotated test split, GRASP-v2 reaches **0.924 ROC-AUC vs 0.770** for CCP, the best published baseline (Δ +0.154, 95% CI +0.110 to +0.199). It wins on all four models and needs no NLI model. On a second benchmark, **RAGTruth**, the same recipe retrained on RAGTruth's own labels reaches **0.827 vs 0.623** (Δ +0.203).
> **Cost, same GPU (T4):** GRASP-v2 costs 33–38% of generation time; CCP costs 192–430%.
> **Against UHeads** (closest prior method, authors' pre-trained heads, same claims): with in-domain labels GRASP-v2 wins on both benchmarks (RAGTruth 0.827 vs 0.680).
> **Main caveats:** GRASP-v2 is *supervised* and needs labels from the setting it monitors. Trained on MUCH and applied to RAGTruth without its labels, it reaches only CCP's level (0.639 with label-free adaptation), and there pre-trained UHeads transfer better (0.680). The training-free variant does not beat CCP on either benchmark.
> Report date: 18 October 2026. Supersedes the earlier report, whose headline numbers (0.8343 vs 0.8251, "6.4% overhead", German "win") did not survive the audit — see §8.

1. Summary

- **The problem:** LLMs state invented facts as fluently as true ones. A useful detector has to score each *claim*, not the whole answer, so a system can flag or withhold exactly the wrong part.
- **The idea:** read *where the model was looking* while it wrote each claim. Three attention statistics — grounding, sink leakage and dispersion — come from one extra forward pass. GRASP-v2 combines them with the token probabilities the model already produced.
- **The result:** a small gradient-boosted classifier on these features beats CCP by a wide margin, on every model, and also improves selective answering: withholding the riskiest 30% of claims cuts the hallucination rate of what remains from 26.4% to 7.2% (CCP: 14.7%).
- **Second benchmark:** on RAGTruth (a new generator, retrieval-augmented tasks and a new labelling scheme), the same recipe retrained on RAGTruth beats CCP by +0.203 ROC. Applied zero-shot from MUCH, it ties CCP.
- **The honest limits:** the winning detector needs labelled training data from the target setting. The MUCH test split was read twice (v1 and v2). The training-free variant does not beat CCP. All are stated in §8.

2. The problem

A language model always answers. Asked for a biography, it may write six fluent sentences of which one is invented, and the invented one carries the same confidence as the rest. Scoring the whole answer ("this paragraph is 70% reliable") does not say *which* sentence to check. What is needed is a score per **atomic claim**:

> *"Marie Curie was born in Warsaw in 1899 and received two Nobel prizes."*
>
> - `Marie Curie was born in Warsaw` → low risk
> - `She was born in 1899` → **high risk** (she was born in 1867)
> - `She received two Nobel prizes` → low risk

MUCH provides exactly this setting: claims are pre-segmented, labelled and evaluated under a fixed protocol. Its authors report that the best existing method, CCP, reaches 0.772 ROC-AUC. That is useful but far from solved, and CCP spends more compute than generation itself did.

3. Existing approaches

Family	Example	Claim-level	Needs labels	Cost	Weakness
Token probabilities	max likelihood, entropy	yes	no	negligible	confidence ≠ correctness; dominated by function words
Meaning-filtered probabilities	**CCP** (Fadeeva et al., 2024)	yes	no	~124% of generation (A100, per MUCH)	an NLI model runs on every token
Multi-sample consistency	SelfCheckGPT, semantic entropy	yes	no	10–20×	consistent false beliefs pass
Supervised internal probes	UHeads/LUH, SAPLMA, SinkProbe	some	yes	low at inference	labels needed; transfer across languages/models is often poor

The closest prior work is **UHeads / LUH** (Shelmanov et al., EMNLP 2025). It is also supervised and claim-level, and also uses attention plus token probabilities, but it trains a transformer head on attention maps. GRASP-v2 uses three interpretable statistics and a small tree model instead. A re-run of the prior-art search (October 2026, `docs/03_novelty_recheck.md`) found no published method that beats CCP on MUCH. GRASP-v2 has not yet been compared head-to-head with UHeads.

4. Method

4.1 Three per-token attention signals

The generated tokens are replayed through the model once (teacher forcing, no sampling). For each generated token, each kept head gives:

- **G — grounding ratio:** the share of non-sink attention that goes back to the *prompt* rather than to the model's own freshly written text. Low grounding means the model is building on its own output instead of retrieving.
- **S — sink leakage:** attention mass dumped on the first token, the transformer's "nothing relevant here" position.

- **P — dispersion:** normalised entropy of the attention row. Peaked attention means one clear source; flat attention means none.

The audit fixed two implementation details. The attention row is now the one that *generated* each token, not the one after it. Token saliency is normalised by the number of later tokens exactly once, not twice.

4.2 Choosing heads without labels

A model has hundreds of heads. For each layer, GRASP keeps the heads whose grounding ratio varies most across tokens on 200 unlabelled generations. A head that always returns the same value cannot tell anything apart. v1 kept one head per layer; **v2 keeps four**.

4.3 From tokens to claims (GRASP-v2)

Each claim becomes one row of features:

1. **Attention statistics:** layers are grouped into 8 relative-depth bins, so models of different depth share one feature space. For every bin, kept head and signal, the claim's content tokens give mean, max, min and a saliency-weighted mean (384 features). Each value is z-scored per (layer, head) on unlabelled data from the same model.
2. **Token probabilities** (7 features): surprisal of the chosen token, entropy, top-1 probability and the top-1/top-2 gap. They come from the top-24 probabilities MUCH ships with every generation, so no extra model runs.
3. **Claim length** (2 features).

A gradient-boosted tree classifier (31 leaves, 800 rounds) maps the 393 features to a hallucination probability. A **training-free** variant, GRASP-v2-free, uses no labels: it averages the per-model rank of mid-depth minimum grounding (sign fixed in advance) with the rank of token surprisal and entropy.

The original training-free score, GRASP-0 (saliency-weighted soft-max over a hand-weighted risk, with a claim-boundary reset), is kept as a reference row.

5. Evaluation protocol

- **Data:** all 8 MUCH generation configs (Llama-3.2-3B, Llama-3.1-8B, Gemma-3-4B, Ministral-8B, each at T = 0.7 and 1.0). Features extracted for all 4,873 generations on Kaggle (2× Tesla T4).
- **Split:** MUCH's own. *Test* is the 200 human-annotated generations (867 claims, 229 hallucinated); *train* is the other 4,673 (19,884 claims). MUCH stores no split field; human annotations are identified by their annotator keys (`an0` , `an1`), which gives exactly 200.
- **MUCH protocol:** −1 is the hallucination class; the trailing EOS chunk is dropped; stopwords are masked for the probability baselines; only claims where GPT-4o and GPT-4.1 agree are scored.
- **Selection:** every choice (feature groups, normalisation, learner, number of heads) was made by 5-fold cross-validation on train, grouped by generation, ranked by the *mean of per-model* ROC-AUCs. Pooled ROC could be inflated by recognising which model wrote a claim, because hallucination rates differ (Gemma ~54% vs Llama ~22%).
- **Test reads:** the selection was frozen to a file before the test split was read. This was the **second** read; the first was v1 (§6.6).

6. Results

6.1 Checkpoint: CCP reproduced

Before scoring GRASP, the published baseline was reproduced from MUCH's shipped signals. `ccp_24_8` with the `product` aggregator gives **ROC 0.7747 / PR 0.6426**, against the published 0.772 / 0.639. Our evaluator also matches MUCH's own code exactly on a shared fixture.

6.2 Main result — human-annotated test split (867 claims)

Method	Labels?	ROC-AUC [95% CI]	PR-AUC	TPR @ 10% FPR	ΔROC vs CCP [95% CI]
CCP (`ccp_24_8`)	no	0.770 [0.724, 0.817]	0.637	0.537	—
GRASP-0 (original)	no	0.629 [0.569, 0.687]	0.388	0.188	−0.141 [−0.215, −0.072]
GRASP-v2-free	no	0.764 [0.713, 0.815]	0.501	0.354	−0.006 [−0.056, +0.041]
GRASP-v2	**yes**	**0.924 [0.896, 0.948]**	**0.845**	**0.790**	**+0.154 [+0.110, +0.199]**
GRASP-v2 + CCP as input	yes	0.920 [0.895, 0.944]	0.838	0.729	+0.150 [+0.111, +0.191]

Bootstrap: 2,000 samples resampled by generation; ΔROC is paired on identical claims. Adding CCP as an input does not help, so its NLI step can be dropped entirely.

6.3 Per model

Model (test claims)	CCP	GRASP-v2	ΔROC [95% CI]	PR: CCP → GRASP-v2
Gemma-3-4B (134)	0.726	**0.917**	+0.191 [+0.073, +0.284]	0.793 → 0.911
Llama-3.1-8B (282)	0.831	**0.937**	+0.106 [+0.046, +0.155]	0.682 → 0.871
Llama-3.2-3B (279)	0.767	**0.889**	+0.122 [+0.047, +0.193]	0.519 → 0.728
Ministral-8B (172)	0.830	**0.897**	+0.067 [+0.013, +0.128]	0.712 → 0.815

6.4 Transfer to a model the detector has never seen

The classifier is fitted on the training claims of three models and scored on the test claims of the fourth:

Unseen model	GRASP-v2	CCP
Gemma-3-4B	**0.808**	0.726
Llama-3.1-8B	**0.882**	0.831
Llama-3.2-3B	0.759	**0.767**
Ministral-8B	**0.853**	0.830

So the detector stays comparable to or better than CCP even without labels from the target model. In v1 it lost on all four.

6.5 What each ingredient contributes (train cross-validation, mean per-model ROC)

Features (same learner: gradient boosting, depth 3)	1 head / layer	4 heads / layer
Token probabilities only	0.771	0.771
Attention only	0.811	0.832
Attention + token probabilities	0.854	0.866
… + claim length	0.857	0.867
… tuned learner (31 leaves, 800 rounds)	**0.881**	**0.885**
CCP, same claims		*0.797*

Attention alone already beats CCP, and token probabilities add information it lacks. More heads help consistently. Per-model rank normalisation helps logistic regression but not the tree model.

6.6 How the result evolved (and what was corrected)

Stage	Test claims	Best GRASP	CCP	Note
Pre-audit report	727 (one config, no split)	0.834 (fused)	0.825	invalid: no train/test split, test-selected aggregator, misaligned attention
Audited, one config	136	0.737 (fused)	0.776	GRASP-0 alone 0.635; sample too small
v1 (first test read)	867	0.891 (probe + CCP)	0.770	supervised; no cross-model transfer
v2 (second test read)	**867**	**0.924 (no CCP)**	**0.770**	**4 heads + token probabilities; transfers**

6.7 Runtime, measured on the same GPU

Model	tokens / answer	generation	GRASP-v2	overhead
Gemma-3-4B	8.3	0.75 s	0.28 s	37%
Llama-3.2-3B	21.4	1.11 s	0.38 s	34%
Llama-3.1-8B	23.9	2.28 s	0.83 s	36%
Ministral-8B	14.9	1.40 s	0.53 s	38%

Generation is timed with MUCH's own generation code; GRASP's teacher-forced eager-attention pass runs on exactly those outputs (2× Tesla T4, model loading excluded). MUCH's own CCP implementation was then timed on the same GPU type:

Setting	CCP s / token	generation s / token	CCP overhead	GRASP-v2 overhead
MUCH, Gemma-3-4B	0.222	0.090	~245%	37%
MUCH, Llama-3.1-8B	0.228	0.095	~240%	36%
MUCH, Llama-3.2-3B	0.225	0.052	~430%	34%
MUCH, Ministral-8B	0.220	0.094	~235%	38%
RAGTruth, Mistral-7B (195-token answers)	**0.204**	**0.106**	**192%**	**33%**

On the same hardware, GRASP-v2 costs roughly one-sixth to one-twelfth of CCP. CCP's NLI cost is fixed per token, so it hurts most on small, fast models. CCP and generation were timed on different sample generations, so overheads compare per token.

6.8 Second benchmark: RAGTruth

To check the result on data never used in development, the frozen recipe was applied to **RAGTruth** (Niu et al., ACL 2024): responses from Mistral-7B-Instruct to retrieval-augmented QA, summarisation and data-to-text prompts, with human-annotated hallucination spans. Claims come from MUCH's segmenter; a claim is hallucinated if it overlaps an annotated span. The test set is 445 responses and 17,226 claims, 1,323 of them hallucinated. The evaluation rules were fixed before any RAGTruth feature was extracted.

Method (claim level)	Trained on	ROC-AUC [95% CI]	PR-AUC	TPR @ 10% FPR	ΔROC vs CCP [95% CI]
CCP	—	0.623 [0.602, 0.644]	0.150	0.268	—
mean token entropy	—	0.623	0.114	0.178	−0.000 [−0.028, +0.030]
GRASP-v2-free	—	0.595 [0.573, 0.616]	0.098	0.135	−0.028 [−0.057, +0.001]
GRASP-v2, zero-shot	MUCH only	0.625 [0.606, 0.645]	0.118	0.187	+0.002 [−0.025, +0.029]
GRASP-v2, same recipe	**RAGTruth train**	**0.827 [0.803, 0.849]**	**0.383**	**0.556**	**+0.203 [+0.177, +0.233]**

By task, the RAGTruth-trained detector beats CCP everywhere (QA 0.859 vs 0.626, summary 0.765 vs 0.617, data-to-text 0.853 vs 0.624). The zero-shot MUCH-trained detector is better than CCP on QA (0.677) and summaries (0.652) but worse on data-to-text (0.568). At response level (max over claims) the RAGTruth-trained detector scores 0.832 vs CCP's 0.647.

> **What RAGTruth shows:** the recipe generalises (it wins again when trained on the new setting's labels), but a detector trained on one benchmark does not transfer to a different task family and labelling scheme without retraining.

6.9 Label-free adaptation, response level, and UHeads

Label-free adaptation. Three ways to adapt the MUCH-trained detector without target labels were compared on MUCH's training split (leave one model out). Per-domain rank normalisation of every feature won: mean

held-out ROC 0.819 vs 0.808, better on all four models. Applied once to RAGTruth it lifts zero-shot ROC from 0.625 to 0.639. That is closer to, but still not significantly above, CCP's 0.623 (Δ +0.015, CI −0.011 to +0.042).

Response level. Does an answer contain any hallucination? On MUCH's 200 test answers, GRASP-v2 scores 0.911 vs CCP's 0.762 (Δ +0.149, CI +0.086 to +0.214). The training-free variants lose (best: 0.644), so the training-free response-level edge seen on RAGTruth does not hold on MUCH.

UHeads. The authors' pre-trained claim-level heads (Shelmanov et al., EMNLP 2025), run on exactly the same claims. Score sign and head choice were fixed on training claims only.

Benchmark	CCP	UHead (pre-trained)	GRASP-v2, no target labels	GRASP-v2, in-domain
MUCH, Llama-3.1-8B (282 claims)	0.831	0.570	0.883	**0.937**
RAGTruth, Mistral (17,181 claims)	0.623	**0.680**	0.639	**0.827**

With in-domain labels GRASP-v2 beats UHeads on both benchmarks (RAGTruth Δ +0.147, CI +0.116 to +0.180). Without them, UHeads' pre-trained head transfers to RAGTruth better than GRASP trained on MUCH alone (Δ −0.041, CI −0.076 to −0.004). It was trained on far larger, more varied data. The pre-trained Llama head is weak on MUCH even on training claims, likely because MUCH's claims are very short (~4.6 tokens), so the MUCH margin over UHeads is indicative only.

7. Topic 16: uncertainty-aware abstention

A detector earns its keep when the system can *withhold* the claims it doubts. Ranking test claims by risk and withholding the riskiest share gives the following:

Withhold	CCP: hallucinations in kept claims	GRASP-v2-free	GRASP-v2
nothing	26.4%	26.4%	26.4%
riskiest 10%	20.5%	23.1%	**18.9%**
riskiest 20%	16.3%	19.9%	**11.4%**
riskiest 30%	14.7%	16.0%	**7.2%**
Area under risk–coverage (lower is better)	0.152	0.127	**0.062**

In practice, a system that withholds or routes for verification the 30% of claims GRASP-v2 rates riskiest keeps 70% of its claims at a 7.2% hallucination rate, against 14.7% with CCP. Because scoring is one forward pass plus a tree model, this works at claim granularity with modest latency.

8. Limits and corrections

1. **Supervision.** GRASP-v2 is trained on MUCH's training labels (GPT-4o annotations). The fully training-free GRASP-v2-free ties CCP on ROC (Δ −0.006, CI includes zero) but is weaker on PR and TPR.
2. **Two test reads.** v1 and v2 both read MUCH's test split. All v2 choices were frozen on train first. The RAGTruth check (§6.8), whose rules were fixed in advance, confirms the in-domain result but shows no zero-shot advantage.
3. **Labels.** Test claims are scored against GPT-4o labels on the human-annotated samples, as MUCH's evaluator does, not against the human labels.
4. **Optimism.** Test ROC (0.924) sits above train cross-validation (~0.885–0.897). The final model trains on 100% of train and the test set is small, so expect some regression on new data. The lower CI bound is 0.896.
5. **Transfer.** Trained on MUCH and applied to RAGTruth without retraining, GRASP-v2 reaches only CCP's level (0.639 with label-free adaptation), and pre-trained UHeads do better there (0.680).
6. **Prior work.** No head-to-head comparison with UHeads/LUH yet.

Corrections to the earlier report. An independent audit (`audit_report.md`) found the earlier evaluation had no train/test separation, selected the fusion aggregator on test data, compared a single-model German subset against CCP's 4-model pooled score, used attention rows shifted by one token, normalised saliency twice, and printed the soft-min formula with inverted signs. It also claimed 1–3% overhead against an estimated, not measured, denominator. All of these are fixed, and the numbers above replace the earlier 0.8343 vs 0.8251, “6.4% overhead” and “beats CCP on German” claims, which should not be cited.

9. Next steps

- **Transfer without labels:** rank normalisation narrowed the gap but did not close it. Training on more diverse data, as UHeads does, is the likely route.
- **UHeads retrained on MUCH**, to compare both methods with identical training data, rather than using their released heads.

10. Reproducibility

Code: `src/grasp/` (features, head selection, scoring, MUCH protocol) and `scripts/01–14`. GPU extraction: `kaggle/build_notebook.py`, which builds a self-contained Kaggle notebook. Development and frozen choices: `scripts/12_develop_v2.py` → `results/dev/frozen_v2_*.json`. Test evaluation: `scripts/13_final_eval_v2.py` → `results/final_eval_v2.json`. Abstention and per-claim test scores: `scripts/14_abstention_v2.py` → `results/test_scores_v2.csv`. RAGTruth: `scripts/15_ragtruth_extract.py` (Kaggle) and `scripts/16_ragtruth_eval.py` → `results/ragtruth_eval.json`. Full write-up: `results/FINAL_RESULTS_v2.md`. Unit tests: 20 in `src/tests/test_grasp.py`, 10 protocol checks in `src/tests/test_much_eval.py`.